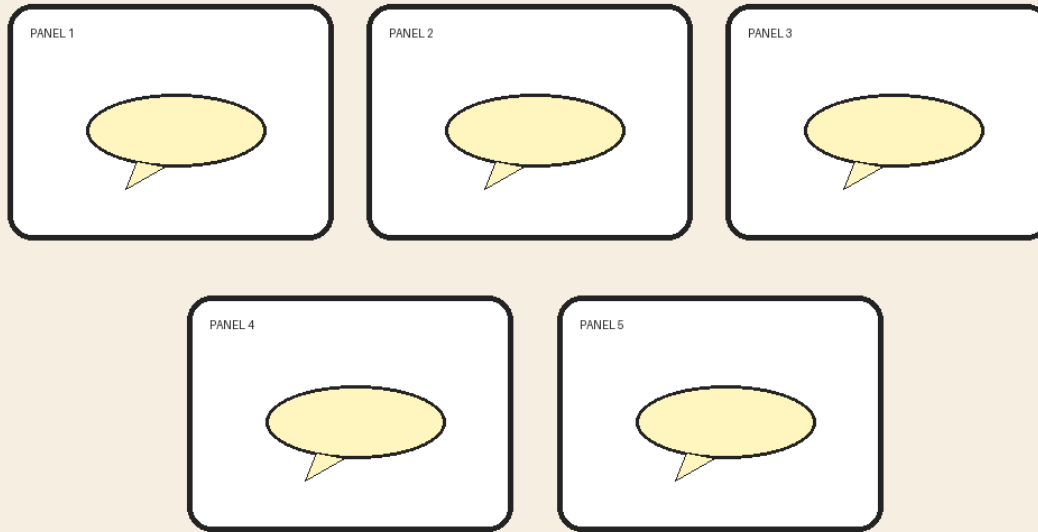


COMICCRAFT



AI Comic Story Creator using Gemini Models

PROJECT REPORT

ComicCraft – AI Comic Story Creator using Gemini Models

A web-based AI application for personalized comic story, dialogue, illustration, preview, and PDF export.

1. Project Description

ComicCraft is a web-based application that uses AI to generate personalized comic book stories and illustrations from user-provided prompts. It is built with FastAPI and integrates Google Gemini models with Stable Diffusion.

The application accepts a story prompt, main character name, setting, tone, and art style, then produces a panel-by-panel storyline with corresponding illustrations. Users can preview the comic and download the complete comic as a PDF.

2. Scenarios

Scenario 1 – A user enters a story such as “A brave fox exploring an enchanted forest.” The system generates a multi-panel comic with text and images.

Scenario 2 – The user changes the tone to funny and the art style to comic book, causing the pipeline to regenerate the outline, story, and images.

Scenario 3 – After reviewing the preview, the user downloads the comic. The layout builder assembles panel content and the PDF exporter creates the final document.

3. Technical Architecture

Frontend: HTML, CSS, and Jinja2 templates.

Backend: FastAPI for routing, input processing, AI workflow coordination, layout building, and PDF export.

AI Integration: Gemini Flash for structured panel outlines, Gemini Pro for detailed narration and dialogue, and Stable Diffusion for comic-style illustrations.

4. Model Selection

Gemini Flash (models/gemini-1.5-flash): selected for fast and structured generation of comic panel outlines.

Gemini Pro (models/gemini-1.5-pro): selected for detailed comic narration and character dialogue.

Stable Diffusion (runwayml/stable-diffusion-v1-5): selected for generating comic-style illustrations from image prompts.

5. Project Workflow

Milestone 1: Model selection, architecture definition, and development environment setup.

Milestone 2: Core functionality development and FastAPI backend implementation.

Milestone 3: Main application logic and routes.py development.

Milestone 4: Responsive frontend development with HTML, CSS, and Jinja2.

Milestone 5: Local deployment, testing, and verification.

6. Core Functionalities

`generate_outline()` creates a structured five-panel outline.

`generate_story()` expands the outline into narration and character dialogue.

`generate_image()` creates a comic-style image from an image prompt and saves it to the panels directory.

`build_comic_layout()` matches images with their corresponding panel stories.

`save_pdf()` compiles panel images and narration into a multi-page PDF.

7. FastAPI Routes

/ loads the homepage for entering story details.

/generate processes form input, generates comic panels, and returns the preview.

/generate-comic/json accepts JSON input and returns layout data and the generated PDF path.

/export-success displays confirmation after export.

/test-image allows direct testing of image generation.

8. Frontend

The homepage collects story prompt, character name, setting, tone, and art style.

The comic preview page displays each panel with title, illustration, scene description, captions, and narration.

The export success page confirms that the comic has been created and downloaded and provides a path to create another comic.

9. Deployment

The project uses a Python virtual environment and installs dependencies including FastAPI, Uvicorn, Google Generative AI, Hugging Face Diffusers, FPDF, and related packages.

Sensitive values such as the Gemini API key and Hugging Face API token are stored as environment variables.

The local server is launched with: `uvicorn app.main:app --reload`. The application is accessed at `http://127.0.0.1:8000` and FastAPI documentation at `/docs`.

10. Project Testing & Verification

The supplied project documentation describes local verification of the complete workflow: AI text generation, image creation, layout building, frontend rendering, and PDF export.

Testing should verify valid user inputs, correct route behavior, successful comic generation, image generation, preview rendering, and successful PDF export. Invalid inputs and AI/API failures should be handled gracefully.

11. Conclusion

ComicCraft demonstrates an end-to-end AI-assisted comic creation workflow. Gemini Flash and Gemini Pro provide structured and creative storytelling, while Stable Diffusion produces comic-style illustrations. The FastAPI backend connects these services to an interactive frontend and PDF export pipeline.

The modular architecture can support future enhancements such as user profiles, comic libraries, and multi-page story arcs.

Project Technology Summary

Component	Technology / Purpose
Frontend	HTML, CSS, Jinja2 – user input and comic preview
Backend	FastAPI – routing and application logic
Story Outline	Gemini Flash – structured 5-panel outline
Story & Dialogue	Gemini Pro – narration and character dialogue
Images	Stable Diffusion + Hugging Face Diffusers
PDF Export	FPDF – multi-page comic PDF
Server	Uvicorn – ASGI development server
Language	Python